DLP Lab4: Conditional VAE for Video Prediction

Yi-Hsiang Ho, 111550106

1. Introduction

In this lab, I worked on pose-conditioned video prediction: given the first RGB frame and a pose sequence, the model predicts future RGB frames in an autoregressive way. Since long-horizon rollout easily accumulates errors, the model needs both good reconstruction and stable training.

Instead of spending most effort on designing a brand-new architecture, this assignment is mainly about *making the provided CVAE training reliable and high-performing*. In practice, my final score is dominated by training tricks and schedules. The main parts I focused on are:

- **Teacher forcing schedule**: start with a moderate ratio and decay after a few epochs to help the model transition to self-conditioned rollout.
- **KL annealing**: warm up the KL weight (monotonic works best in my runs) to avoid strong regularization too early.
- **Data sampling tricks**: enable shuffling and use a sliding window to increase temporal diversity per epoch.
- **Inference trick**: use Monte Carlo averaging (multiple latent samples) to reduce variance in predictions.

2. Implementation Details

2.1. Code Structure and Data Pipeline

I mainly worked with four parts in the provided codebase: (1) the training script, (2) the testing/inference script, (3) the dataloader, and (4) the model modules (encoders, latent predictor, and decoder).

In the dataloader (`Dataset_Dance`), frames are sorted by index and organized into sequences. Specifically, training mode uses sliding windows with configurable `stride` for better temporal coverage.

In my experiments, I found that enabling training shuffle and using sliding windows can give more diverse temporal windows per epoch, which helps the model generalize better during autoregressive rollout.

2.2. Training and Testing Protocol

During training, the loop in `training_stage()` iterates over epochs and mini-batches (as shown in Fig. 1). In each epoch, the teacher forcing ratio (tfr) and KL weight β are updated (details are discussed in the next few sections).

```python
def training_stage(self):
    best_psnr = -1
    val_loader = self.val_dataloader()

    for i in range(self.args.num_epoch):
        train_loader = self.train_dataloader()
        adapt_TeacherForcing = True if random.random() < self.tfr else False

        for (img, label) in (pbar := tqdm(train_loader, ncols=120)):
            img = img.to(self.args.device)
            label = label.to(self.args.device)
            loss_dict = self.training_one_step(img, label, adapt_TeacherForcing)
            loss = loss_dict['total_loss']

            # Tensorboard logging (step)
            # ...

            beta = self.kl_annealing.get_beta()

            # tqdm bar update
            # ...

        # Tensorboard logging (epoch)
        # ...

        # Keep training-time validation fast with single-sample rollout.
        mse, psnr = self.eval(val_loader, mc_samples=1)

        # Save bets ckpt and every `per_save` epochs
        # ...

        self.current_epoch += 1
        self.scheduler.step()
        self.teacher_forcing_ratio_update()
        self.kl_annealing.update()

    print(f"Best PSNR: {best_psnr}")
```

Figure 1. **Training stage**: epoch loop, batch loop, and per-epoch updates.

In each batch, `training_one_step()` executes these steps:

- **Encode all frames/poses**.
- **Roll out over time**: for each timestep t, predict latent z_t from encoder (`Gaussian_Predictor`) using encoded image/label features.
- **Teacher forcing decision**: for $t > 0$, the previous input frame is either the ground-truth frame x_{t-1} or the previous prediction $\hat{x}_{t-1}$ (detached). With per-epoch teacher forcing, the decision is fixed for the whole epoch; with scheduled sampling, it is sampled per frame.
- **Decode next frame**: fuse (previous-frame feature, label feature, and z_t) with `Decoder_Fusion`, then generate RGB with `Generator` and `sigmoid`.

I include the corresponding code structure in Fig. 2 for reference.

```python
def training_one_step(self, img, label, adapt_TeacherForcing):  # TODO
    """
    img: (B, T, 3, H, W)
    label: (B, T, 3, H, W)
    """
    B, T, C, H, W = img.shape

    img_reshape = img.reshape(-1, C, H, W)
    label_reshape = label.reshape(-1, C, H, W)

    imgs_enc_all = self.frame_transformation(img_reshape).reshape(B, T, -1, H, W)
    labels_enc_all = self.label_transformation(label_reshape).reshape(B, T, -1, H, W)

    mus = []
    logvars = []
    imgs_pred = []

    for t in range(T):
        z, mu, logvar = self.Gaussian_Predictor(imgs_enc_all[:, t], labels_enc_all[:, t])
        mus.append(mu)
        logvars.append(logvar)

        if t == 0:
            curr_frame_enc = imgs_enc_all[:, 0]
        else:
            if self.args.scheduled_sampling:
                use_tf = True if random.random() < self.tfr else False
            elif adapt_TeacherForcing:
                use_tf = True
            else:
                use_tf = False

            if use_tf:
                prev_img = img[:, t-1]  # Use GT frame
            else:
                prev_img = imgs_pred[-1].detach()  # Use predicted frame
            curr_frame_enc = self.frame_transformation(prev_img)

        latent_fusion = self.Decoder_Fusion(curr_frame_enc, labels_enc_all[:, t], z)
        imgs_pred.append(torch.sigmoid(self.Generator(latent_fusion)))

    imgs_pred = torch.stack(imgs_pred, dim=1)
    mu_stack = torch.stack(mus, dim=1)
    logvar_stack = torch.stack(logvars, dim=1)

    mse_loss = self.reconstruction_loss(imgs_pred, img)
    beta = self.kl_annealing.get_beta()
    kl_loss = kl_criterion(mu_stack, logvar_stack)

    loss = mse_loss + beta * kl_loss

    # Early stopping when training failed
    if not (torch.isfinite(mse_loss) and torch.isfinite(kl_loss) and
torch.isfinite(loss)):raise ValueError(
            f"\nWarning: non-finite loss at Epoch {self.current_epoch}. "
            f"mse={mse_loss.item()}, kl={kl_loss.item()}, beta={beta:.6f}. Stop."
        )

    self.optim.zero_grad()
    loss.backward()
    self.optimizer_step()

    return {
        'total_loss': loss,
        'mse_loss': mse_loss,
        'kl_loss': kl_loss
    }
```

Figure 2. **Training one step**: encoding, latent prediction, teacher forcing, and decoding.

After generating the whole predicted sequence, `training_one_step()` computes a reconstruction term and a KL term:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \beta\, \mathcal{L}_{\text{KL}}. \tag{1}$$

In practice, $\mathcal{L}_{\text{rec}}$ is MSE (with an optional tail-weighting that emphasizes later frames), and β comes from the KL annealing scheduler. If the code detects non-finite losses, it raises an error to stop training early.

In `optimizer_step()`, the code clips gradients and skips the update if it detects any NaN/Inf in gradients (instead of silently applying a bad step). I kept these safeguards because some hyper-parameter settings can easily cause NaNs.

During validation and testing, the model predicts 630 frames per sequence. Since no ground truth of last frame is available, the model always conditions on its own previous prediction $\hat{x}_{t-1}$. To reduce variance from a single latent sample, in testing time, it average multiple latent samples at each step (Monte Carlo sampling). In my best setting, I use `mc_samples=32`. The code snippet is in Fig. 3.

```python
T, B, C, H, W = label.shape

label_reshape = label.reshape(-1, C, H, W)
labels_enc_all = self.label_transformation(label_reshape).reshape(T, B, -1, H, W)

for t in range(1, T):
    prev_img = decoded_frame_list[-1].to(self.args.device)
    curr_frame_enc = self.frame_transformation(prev_img)

    # Monte Carlo inference: sample multiple z and average predictions.
    mc_pred_sum = torch.zeros_like(prev_img)
    for _ in range(self.args.mc_samples):
        z = torch.randn(B, self.args.N_dim, H, W, device=self.args.device)
        latent_fusion = self.Decoder_Fusion(curr_frame_enc, labels_enc_all[t], z)
        mc_pred_sum += torch.sigmoid(self.Generator(latent_fusion))

    pred_frame = mc_pred_sum / self.args.mc_samples
    decoded_frame_list.append(pred_frame.cpu())
    label_list.append(label[t].cpu())
```

Figure 3. Testing logic

2.3. Reparameterization Trick.

In `Gaussian_Predictor.reparameterize()`, latent sampling is implemented as

$$\sigma = \exp\left(0.5 \log \sigma^2\right),\ \epsilon \sim \mathcal{N}(0, I),\ z = \mu + \sigma \odot \epsilon. \tag{2}$$

μ and $\log \sigma^2$ are also clamped for numerical stability, which is helpful when training becomes unstable.

```python
class Gaussian_Predictor(nn.Sequential):
    def __init__():
        #...

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, img, label):
        feature = torch.cat([img, label], dim=1)
        parm = super().forward(feature)
        mu, logvar = torch.chunk(parm, 2, dim=1)

        mu = torch.clamp(mu, -30.0, 30.0)
        logvar = torch.clamp(logvar, -30.0, 20.0)

        z = self.reparameterize(mu, logvar)
```

Figure 4. Reparameterization trick in Gaussian predictor.

2.4. Teacher Forcing Strategy.

I tried the two teacher forcing strategies:

- **Per-epoch teacher forcing**: one Bernoulli decision per epoch (`adapt_TeacherForcing`); all time steps in that epoch follow the same mode.
- **Scheduled sampling**: one Bernoulli decision per frame (enabled by `--scheduled_sampling`).

Teacher forcing ratio is decayed by

$$\text{tfr} \leftarrow \max(0, \text{tfr} - \Delta) \tag{3}$$

after `current_epoch >= tfr_sde`, where Δ is `tfr_d_step`.

2.5. KL Annealing Ratio

The `kl_annealing` class supports:

- **Monotonic**: one linear warm-up from 0 to 1 over the annealing portion of training. Here, `kl_anneal_ratio` decides how large this warm-up region is, and the same linear ramp is applied once from start to end of that region.
- **Cyclical**: the same warm-up idea is repeated in each cycle. Within every cycle, `kl_anneal_ratio` still means the fraction of the cycle used for ramping up β, while the remaining part keeps β at its final value. So compared with monotonic annealing, cyclical annealing is basically a repeated warm-up-and-hold schedule.
- **Fixed**: constant $\beta = 1$ (no annealing).

At initialization, `frange_cycle_linear` is called to calculate the annealing schedule for the whole training process, and saved in `betas`. On each epoch, the current β is updated by indexing into `betas` with the current epoch number.

For robustness, the final beta is lower-bounded by `kl_beta_min`.

```python
class kl_annealing():
    def __init__(self, args, current_epoch=0):
        # Init member variable
        #...
        if self.type == "cyc":
            self.betas = self.frange_cycle_linear(
                n_iter=self.num_epoch,
                n_cycle=self.cycle,
                ratio=self.ratio
            )
        elif self.type == "mono":
            self.betas = self.frange_cycle_linear(
                n_iter=self.num_epoch,
                n_cycle=1,
                ratio=self.ratio
            )
        else:
            self.betas = [1.0] * self.num_epoch

    def update(self):
        self.current_epoch += 1

    def get_beta(self):
        idx = min(self.current_epoch, self.num_epoch - 1)
        return max(float(self.betas[idx]), self.beta_min)

    def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0,  n_cycle=1,
ratio=1)betas = np.ones(n_iter)
        period = n_iter // n_cycle
        step = (stop - start) / (period * ratio)

        for i in range(n_iter):
            i_in_cycle = i % period
            if i_in_cycle < period * ratio:
                betas[i] = start + step * i_in_cycle
            else:
                betas[i] = stop

        return betas
```

Figure 5. KL annealing schedules.

3. Analysis and Discussion

3.1. Teacher Forcing Ratio and Loss Trend

Figure 6 overlays teacher forcing ratio and training total loss in the same figure. Since TFR is logged per epoch while loss is logged per step, I converted loss to an step-aligned horizontal axis for direct comparison. As tfr decays after the scheduled start epoch, the loss starts to increase and fluctuate more, which is expected because the model is now conditioning on its own predictions instead of ground truth. However, after a few epochs of adaptation, the loss starts to decrease again. This pattern suggests that the model is successfully transitioning to self-conditioned rollout, and the initial increase in loss is a sign of the model learning.

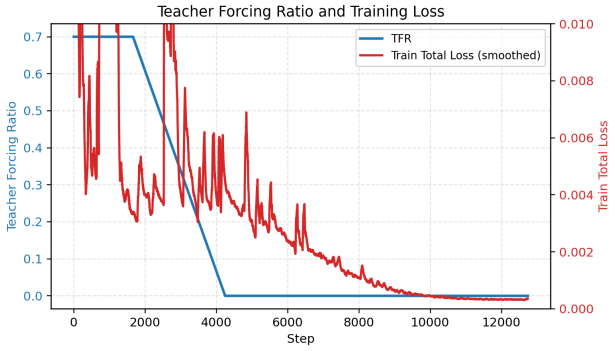


Figure 6. Teacher forcing ratio and training total loss from the same experiment setting.

Figure 7 also uses the same run and shows how validation PSNR changes as teacher forcing ratio decreases. Even though PSNR fluctuates early, it still improves after tfr starts to drop. My interpretation is that once the model gets used to conditioning on its own predictions, it becomes more robust for long-horizon rollout.

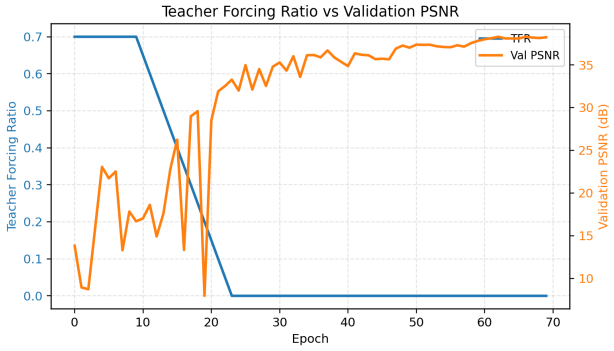


Figure 7. Teacher forcing ratio versus validation PSNR from the same experiment setting.

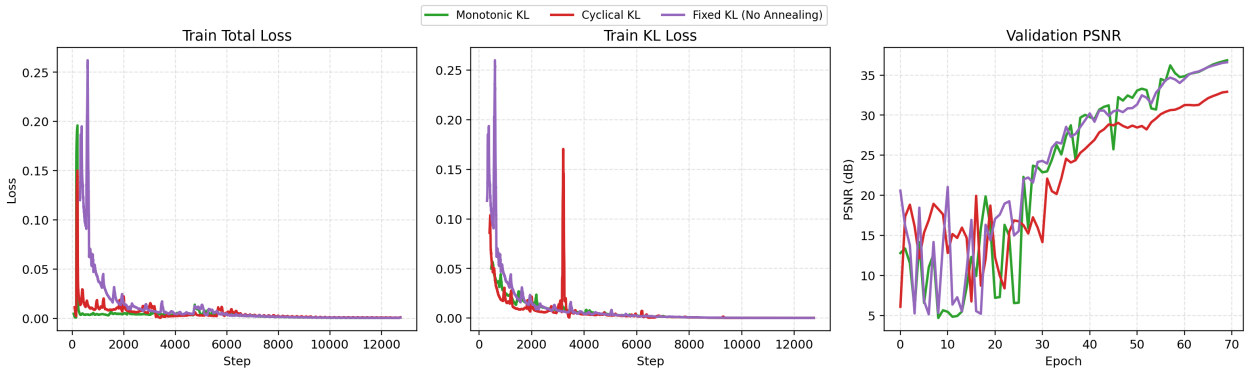


Figure 8. Loss and validation PSNR comparison for monotonic, cyclical, and fixed KL settings.

3.2. Loss Curves Under Different KL Settings

Figure 8 compares three settings: monotonic KL annealing, cyclical KL annealing, and fixed KL ($\beta = 1$, without annealing).

From my runs, monotonic KL gives the best final validation PSNR and a relatively stable training process. Cyclical KL can be competitive, but the KL-related signals oscillate and I found it slightly harder to tune. It needs small learning rate to avoid explosion when the restarting happens. From my experiments, I found that 1e-3 will diverge, 8e-4 will sometimes lead to explosion but can achieve best PSNR when it works among cyclical settings, and 5e-4 is more stable but lower best PSNR. And even when successful training is achieved, the best PSNR is still lower than monotonic KL. Without annealing ($\beta = 1$ from the start), suprisingly, is more easy to train than cyclical KL.

3.3. PSNR Per Frame on Validation Set

Figure 9 shows per-frame PSNR from validation dataset during evaluation. The curve generally decreases with time step, which is expected because autoregressive prediction accumulates small frame-wise errors. However, the curve is not strictly monotonic and has some fluctuations. At around frame 400 to 550, the curve shows a more pronounced decline, which may result from the fact that the motion in that moment is more complex and harder to predict.

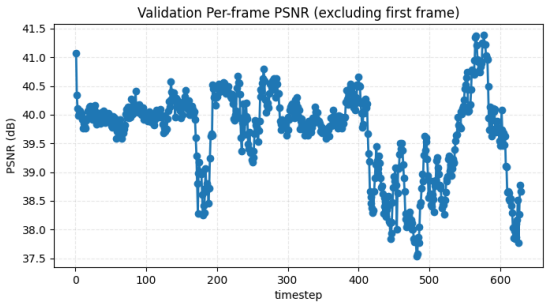


Figure 9. Validation PSNR per frame (from TensorBoard image summary).

3.4. Other Training Strategy Analysis

Data preparation

Shuffling the training dataset and using a sliding window to load data significantly improve the model performance. With the same tfr and KL settings, the best PSNR I got without shuffling and sliding window is around 37.58, while adding shuffling can reach around 38.23 and with both tricks can further improve to around 39.81. I think these strategies increase the diversity of training samples, which helps the model generalize better.

Analysis of TFR Parameters

Beyond the single run in Sec. 3.1, I swept over different combinations of teacher forcing parameters. Table 1 compares per-epoch vs. per-frame and different initial `tfr` values. As shown, per-epoch with `tfr=0.7` gives the best average PSNR. Follow this setting, I further swept over different decay start epochs (`tfr_sde`) and decay steps (`tfr_d_step`). The results are summarized in Tab. 2. It shows that:

- `tfr_sde=10, tfr_d_step=0.05` is the best setting

- Larger `tfr_d_step=0.10` often fails (top-right cells)

Configuration	Avg. PSNR
Per-epoch (`tfr=0.7`)	**35.0**
Per-epoch (`tfr=1.0`)	33.1
Per-frame (`tfr=0.7`)	30.8
Per-frame (`tfr=1.0`)	31.6

Table 1. **Effect of TFR strategy and initial ratio.** The max PSNR is annotated in bold

	`d_step=0.05`	`d_step=0.10`
`tfr_sde=10`	**37.56**	failed
`tfr_sde=15`	36.54	failed
`tfr_sde=20`	36.93	36.16
`tfr_sde=25`	31.86	37.16
`tfr_sde=30`	35.52	36.29

Table 2. **TFR decay parameters: decay start epoch and decay step.** The max PSNR is annotated in bold.

Monte Carlo Averaging at Inference

As mentioned in Sec. 2.2, I used multiple latent samples at each step during inference to reduce variance in predictions. In my experiments, I found that increasing the number of Monte Carlo samples from 1 to 32 can give a slight improvement in PSNR.

4. Conclusion

In this lab, I learned that the model quality is very sensitive to training curriculum. In my experiments, the best result comes from using shuffled training windows, stride 8, monotonic KL annealing, and the teacher forcing schedule `tfr=0.7, tfr_sde=10, tfr_d_step=0.05`. For the final sampling stage, `mc_samples=32` gives a noticeable improvement by reducing sampling variance.

Overall, teacher forcing decay and KL annealing help the model transition from easy (ground-truth conditioned) training to harder (self-conditioned) rollout. However, I still see a clear PSNR drop in later frames, so long-horizon drift is still the main limitation of my current implementation.